Machine Learning

Essential Steps in the Machine Learning Workflow

Class details

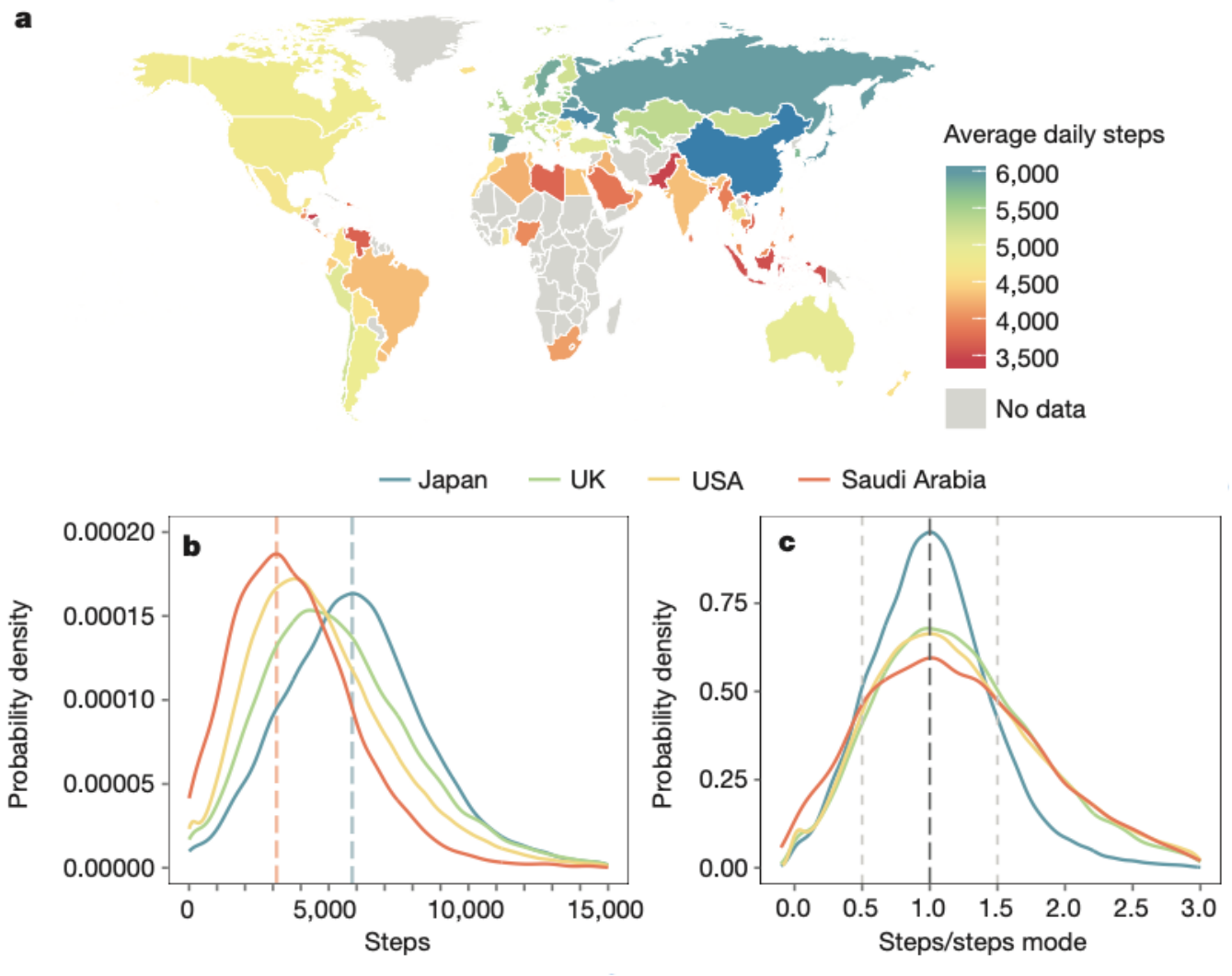
- Any questions about the homework?
- Python?
- Jupyter?
- Topics from last time?

Final Project

- Details posted on Canvas

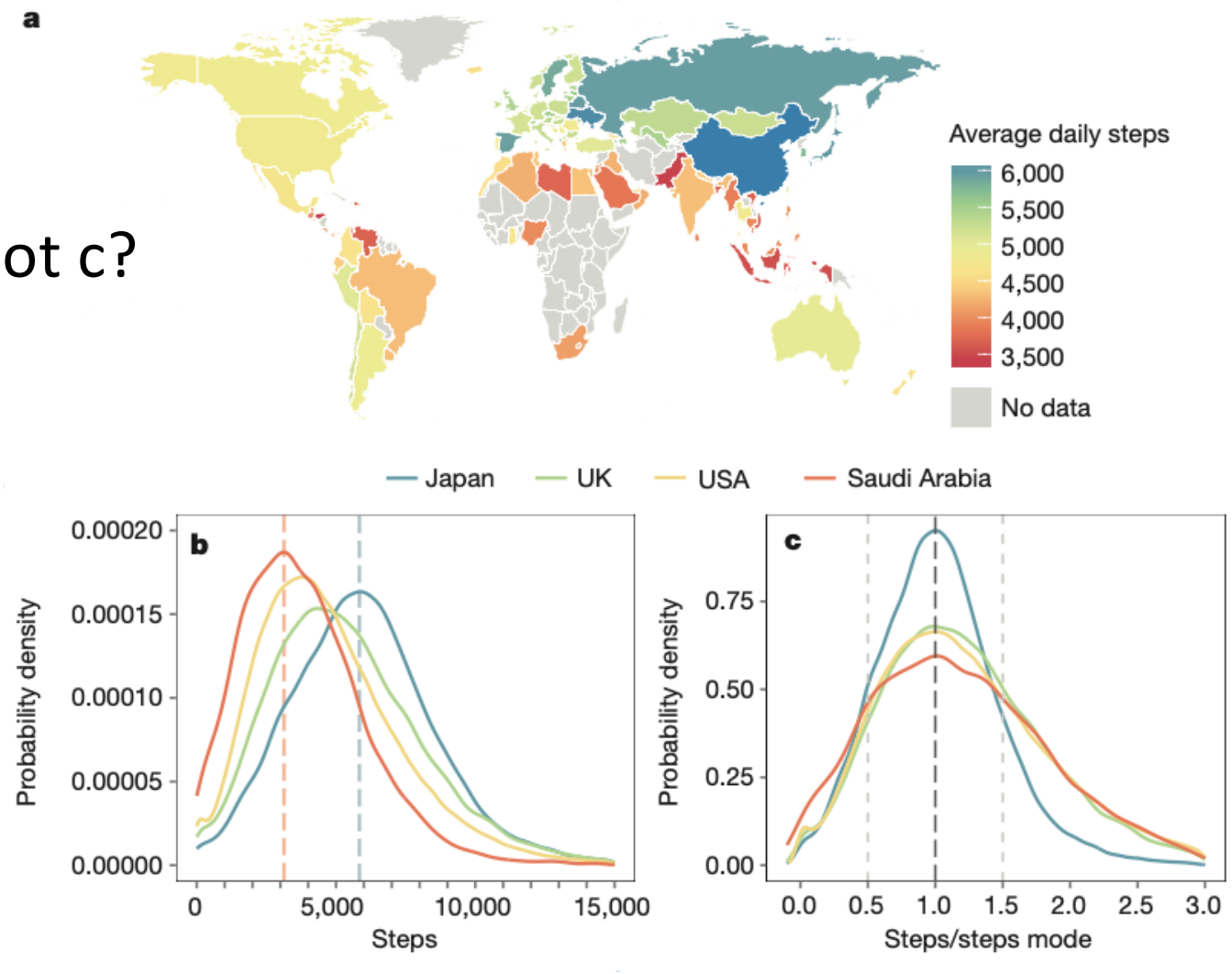
Readings: Technical Notes

- Distributions
  - Modes



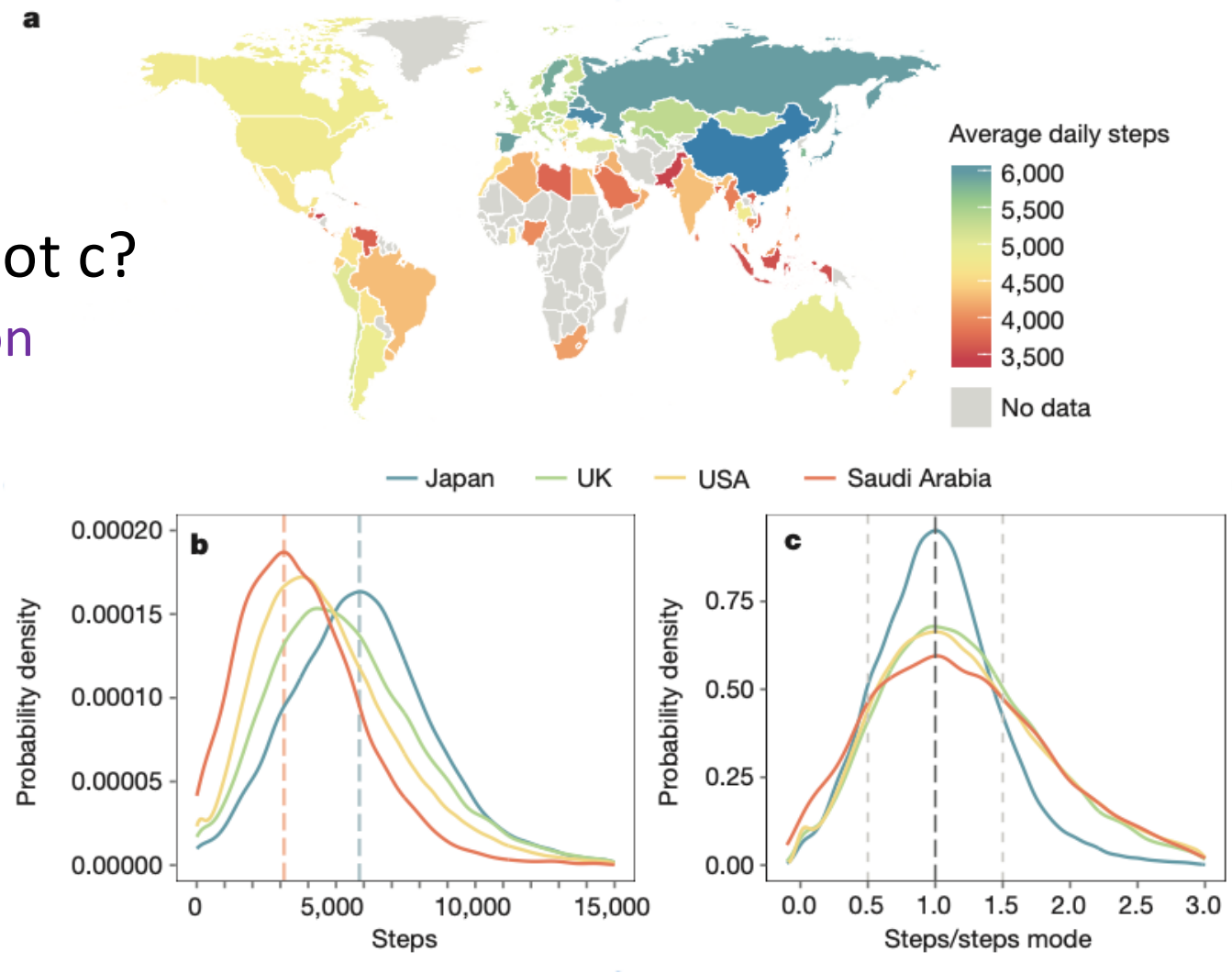
Readings: Technical Notes

- Distributions
  - Modes
- Why did they make plot c?



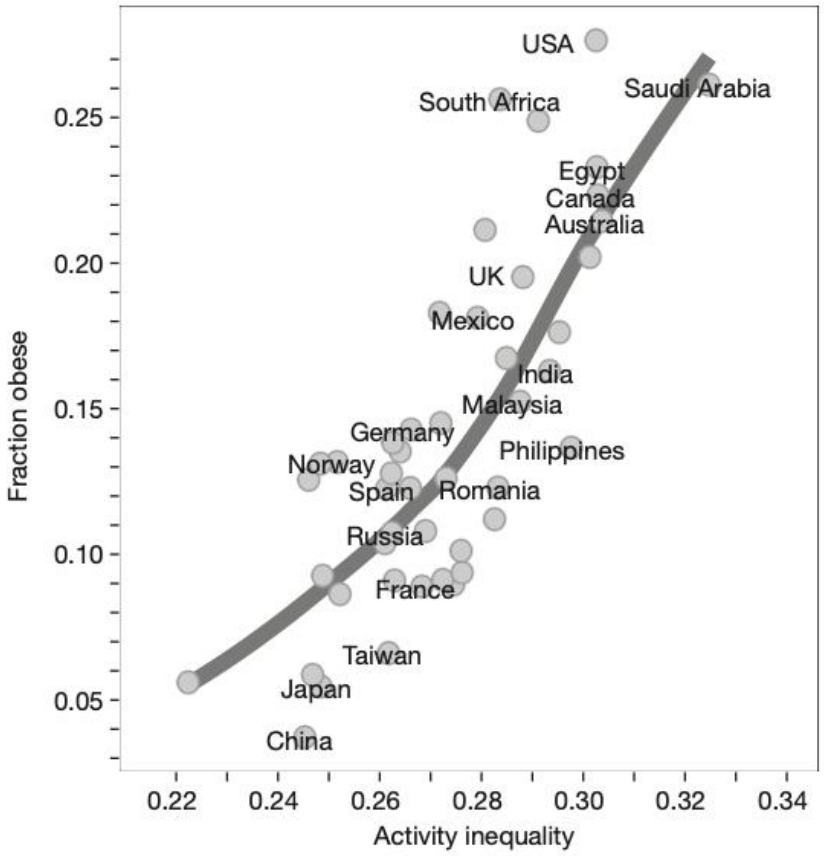
Readings: Technical Notes

- Distributions
  - Modes
- Why did they make plot c?
  - Tails of the distribution



Readings: Technical Notes

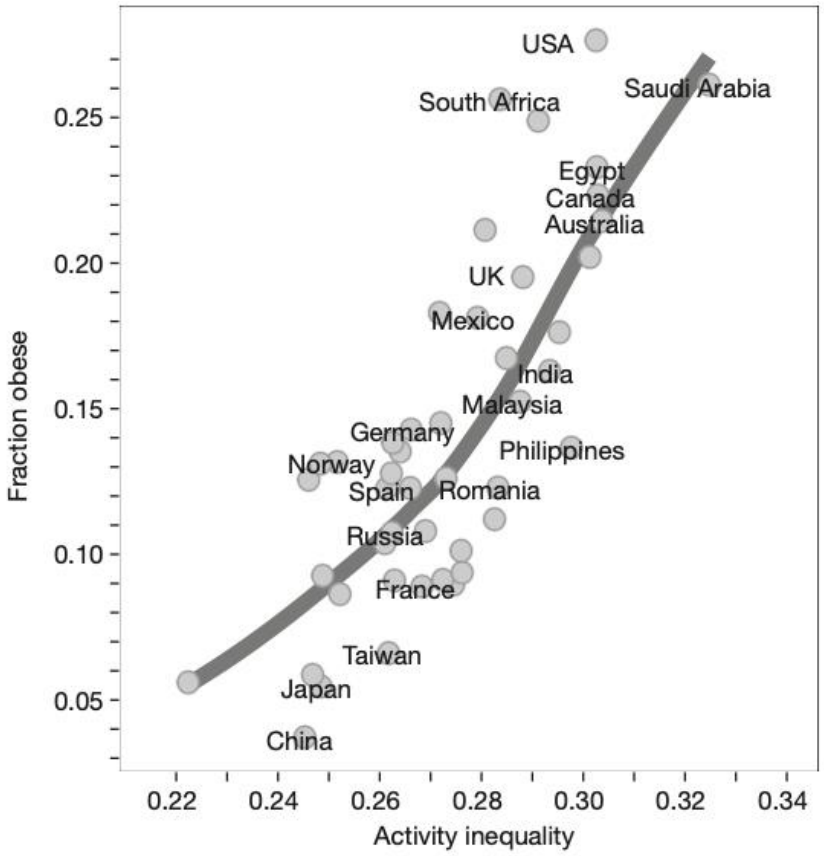
- Distributions
  - Modes
- Why did they make plot c?
  - Tails of the distribution
- Models and fits
  - LOESS: Locally Estimated Scatterplot Smoothing



Readings: Technical Notes

- Distributions
  - Modes
- Why did they make plot c?
  - Tails of the distribution
- Models and fits
  - LOESS: Locally Estimated Scatterplot Smoothing
  - Re-descending M-estimator and Tukey's biweight function

$$x^* = \frac{\sum_{i=1}^{n} w_i x_i}{\sum_{i=1}^{n} w_i}$$



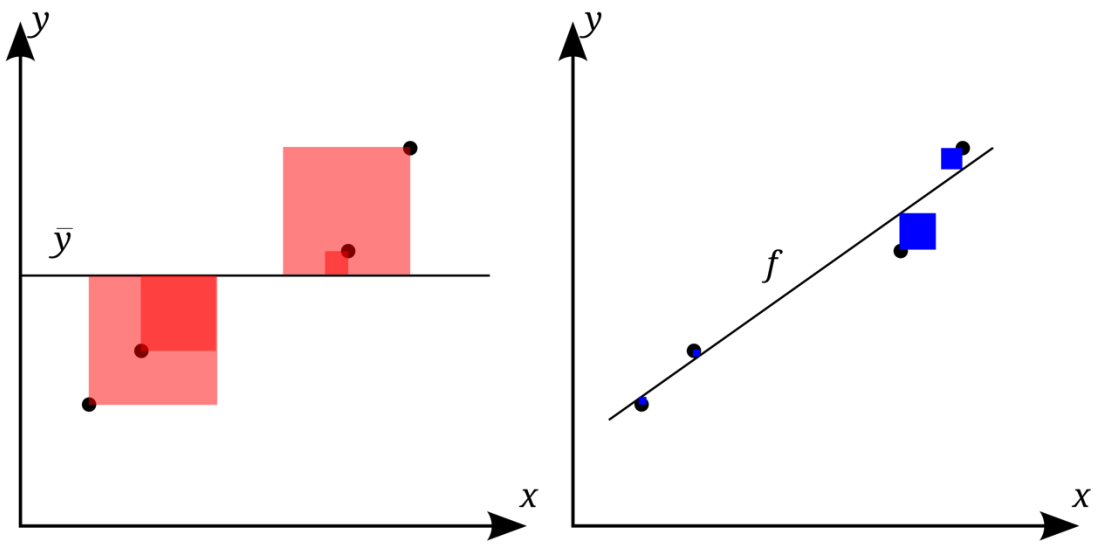
Readings: Technical Notes

- Distributions
- LOESS
- R^2 / ROC / AUC

Readings: Technical Notes

- Distributions
- LOESS
- R^2 : Measuring the performance of a regression model

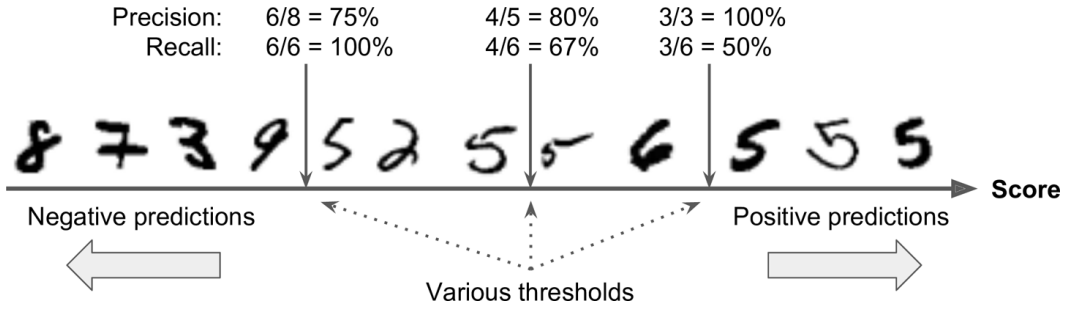
$$R^2 = 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}$$

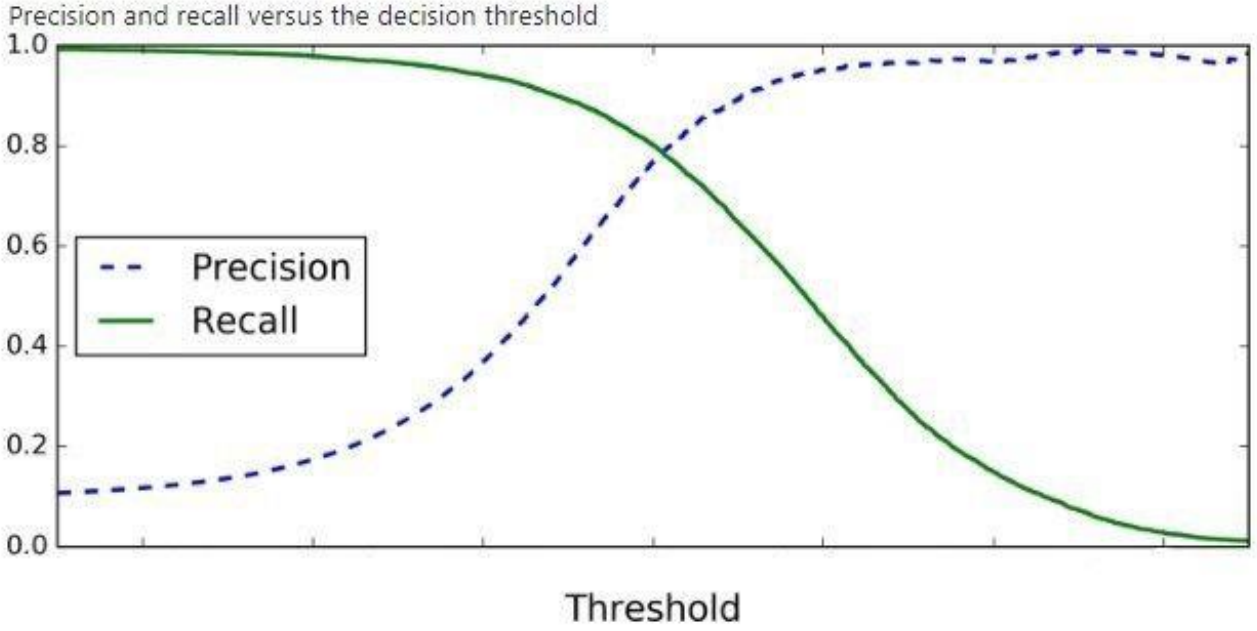


Readings: Technical Notes

- Distributions
- LOESS
- ROC / AUC : Measuring the performance of a ***classification*** model

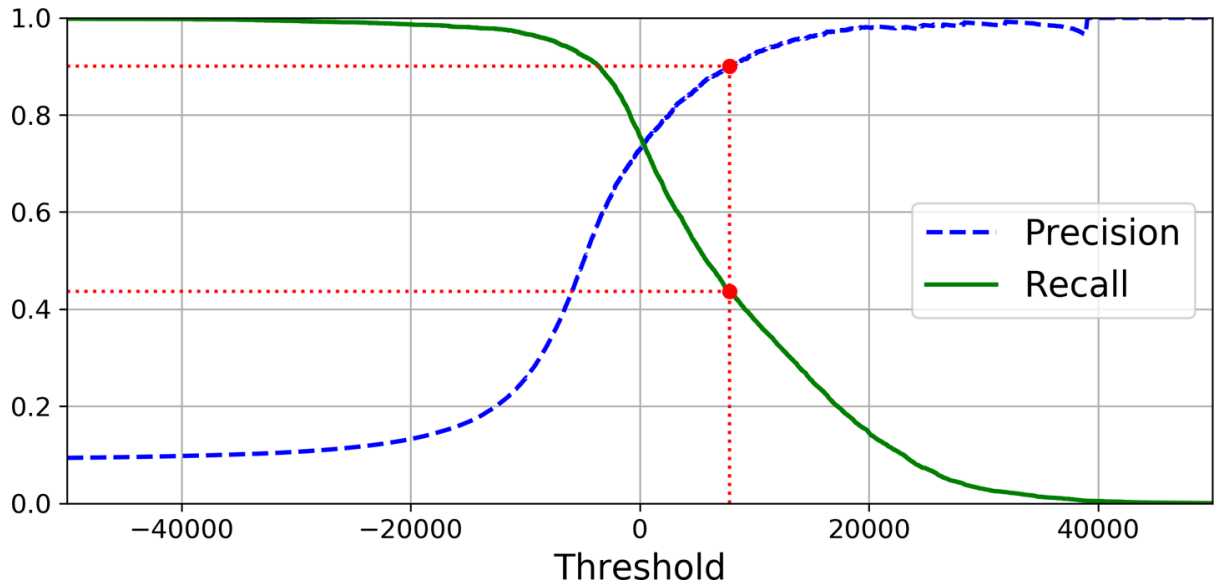
Precision / Recall Trade-off

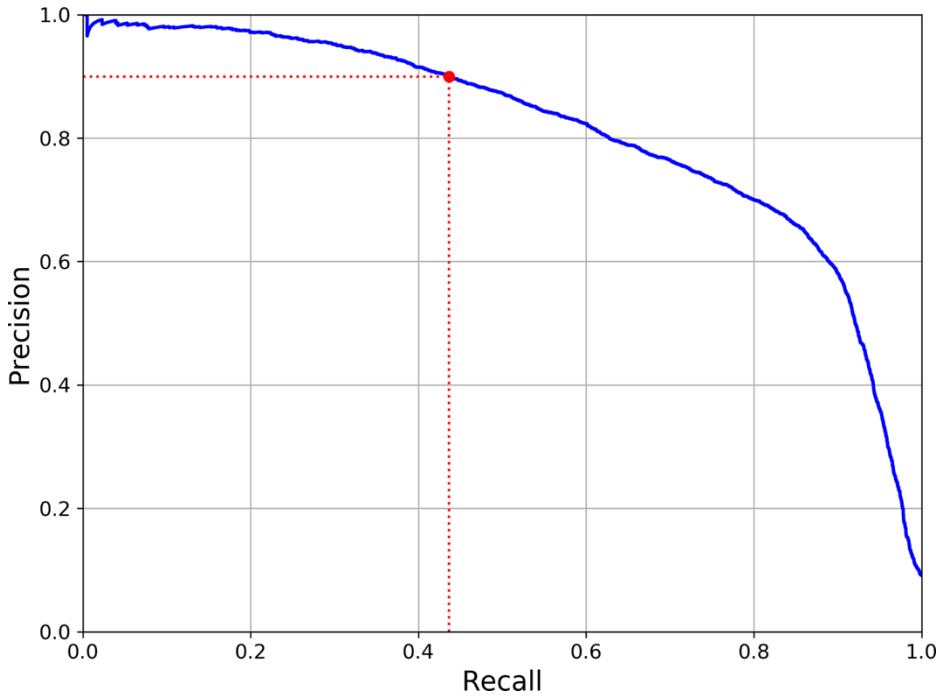




Aurelien Geron's "Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow”

Example of how one might intentionally change the threshold to favor precision vs recall





Aurelien Geron's "Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow”

Recall

Predicted Class (is the alarm going off?)

Actual Class

	No fire	Your house is on fire
No fire	10000	4
Your house is on fire	3	5

True Positive: 5
True Negative: 10000
False Positive: 4
False Negative: 3
N: 10012

Accuracy: (TP + TN) / N = (10005)/10012 = 99.93%

Precision: Accuracy when the alarm is going off
TP / (TP + FP) = 5 / 9 = 55.55%
4 of 9 alarms are false alarms

Recall: Accuracy when your house is actually on fire
TP / (TP + FN) = 5 / 8 = 62.5%
3 of 8 times that your house is on fire, the alarm is not going off

Recall

Predicted Class (is the alarm going off?)

Actual Class		No fire	Your house is on fire
	No fire	10000	4
	Your house is on fire	3	5

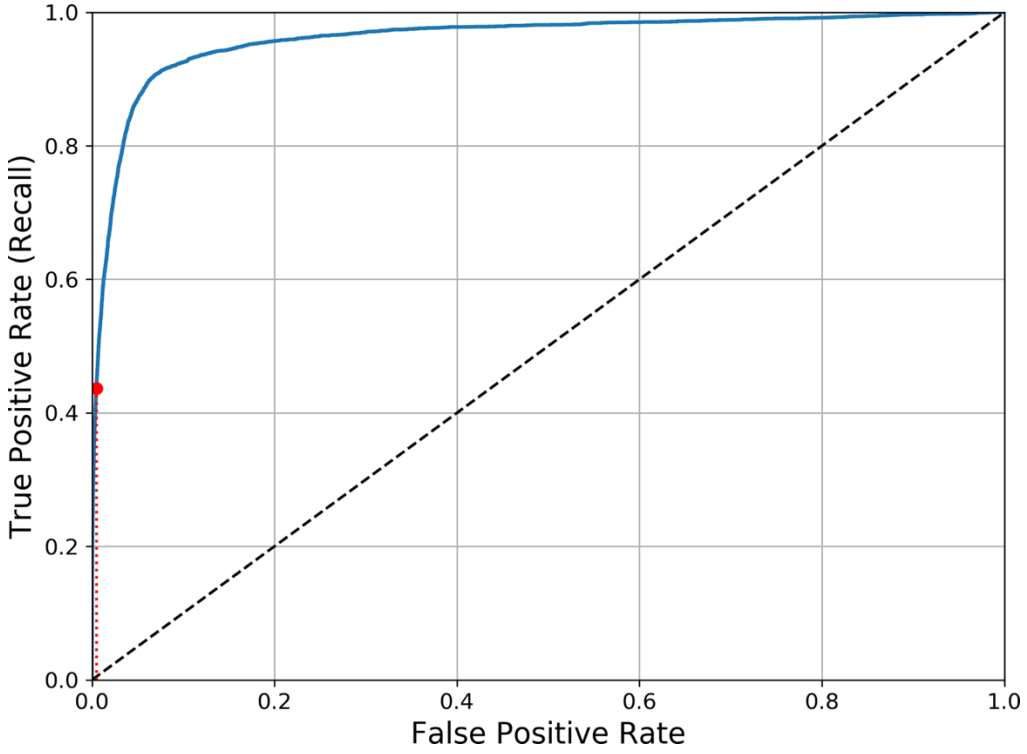
True Positive: 5
True Negative: 10000
False Positive: 4
False Negative: 3
N: 10012

True Positive Rate
Recall: Accuracy when your house is actually on fire
TP / (TP + FN) = 5 / 8 = 62.5%
5 of 8 times that your house is on fire, the alarm is going off

False Positive Rate
Accuracy when your house is NOT on fire
FP / (FP + TN) = 4 / 10004 = 0.04%
4 of 10004 times that your house is on NOT on fire, the alarm is going off

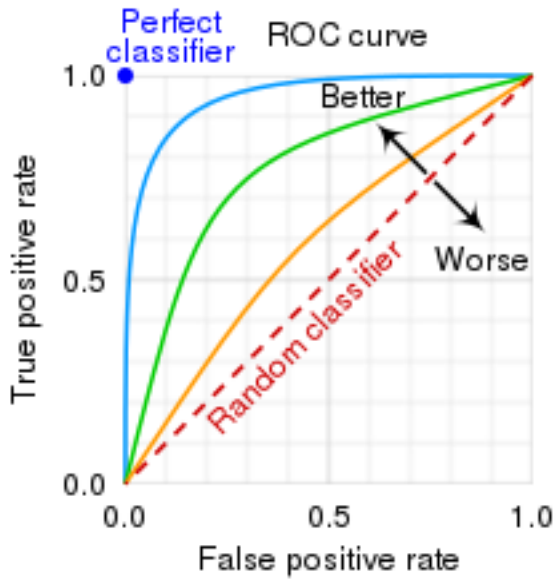
Another common tool: ROC

- ROC: Receiver operating characteristic curve
  - Instead of plotting precision vs recall, ROC plots recall vs false positive rate
- Another trade-off:
  - higher recall -> more false positives
- AUC: Area under the curve
  - 1 for perfect classifier
- There is still room to move around the curve based on threshold value



Readings: Technical Notes

- Distributions
- LOESS
- ROC – plot of the true positive rate vs false positive rate
  - Useful in determining how good a classifier is
- AUC -> area under the curve
  - Best: area = 1
  - Random: area = 0.5



Readings: Technical Notes

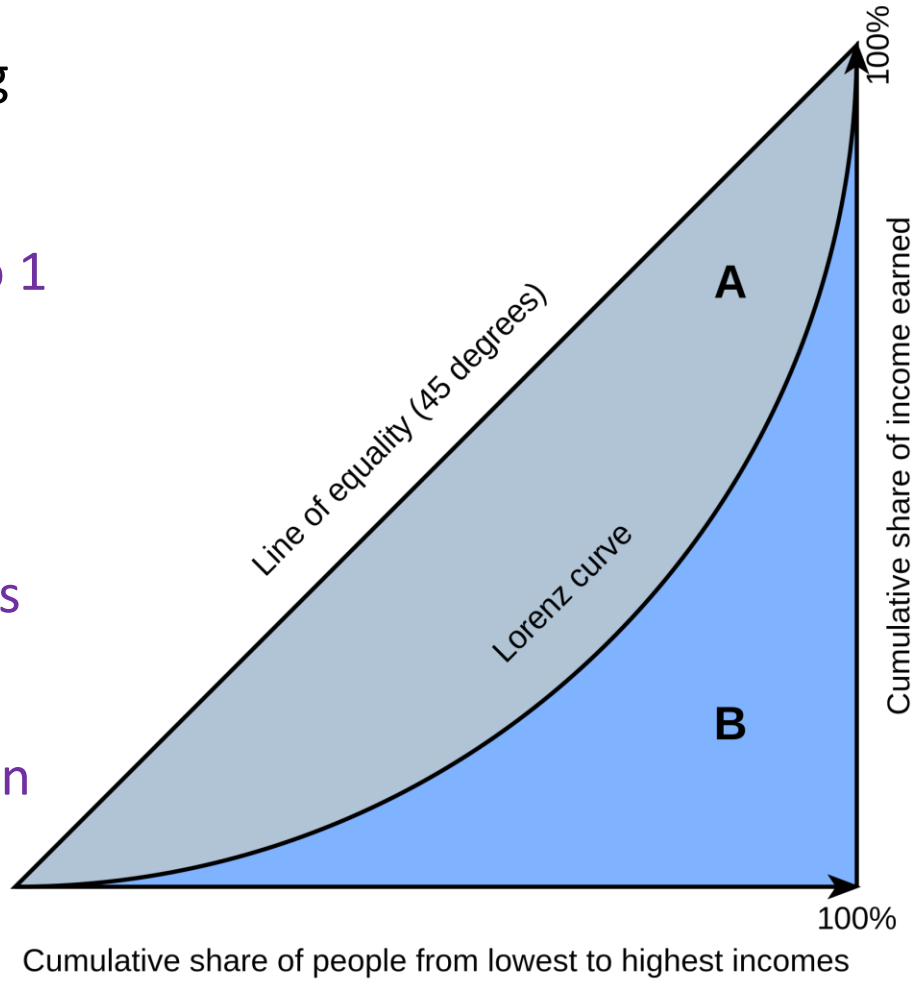
- Distributions
- LOESS
- R^2 / ROC / AUC

Readings: Technical Notes

- Distributions
- LOESS
- R^2 / ROC / AUC
- Gini coefficient

The Gini Coefficient / Lorenz Curve
Different concept than AUC, but similar

- Gini coefficient is a measure representing inequality
  - For example: income inequality
  - Ranges from 0 (complete equality) to 1 (complete inequality)
- We will come across another "Gini" measure next week: the Gini impurity measure
  - For classification, this measure ranges from 0 (totally pure) to 0.5 (totally impure)
  - An entropy measure from information theory



https://en.wikipedia.org/wiki/File:Economics_Gini_coefficient.svg

Readings: Technical Notes

- Distributions
- LOESS
- R^2 / ROC / AUC
- Gini coefficient
- Cross-validation (today) – method for determining hyperparameters, and how well your model can be generalized to new data

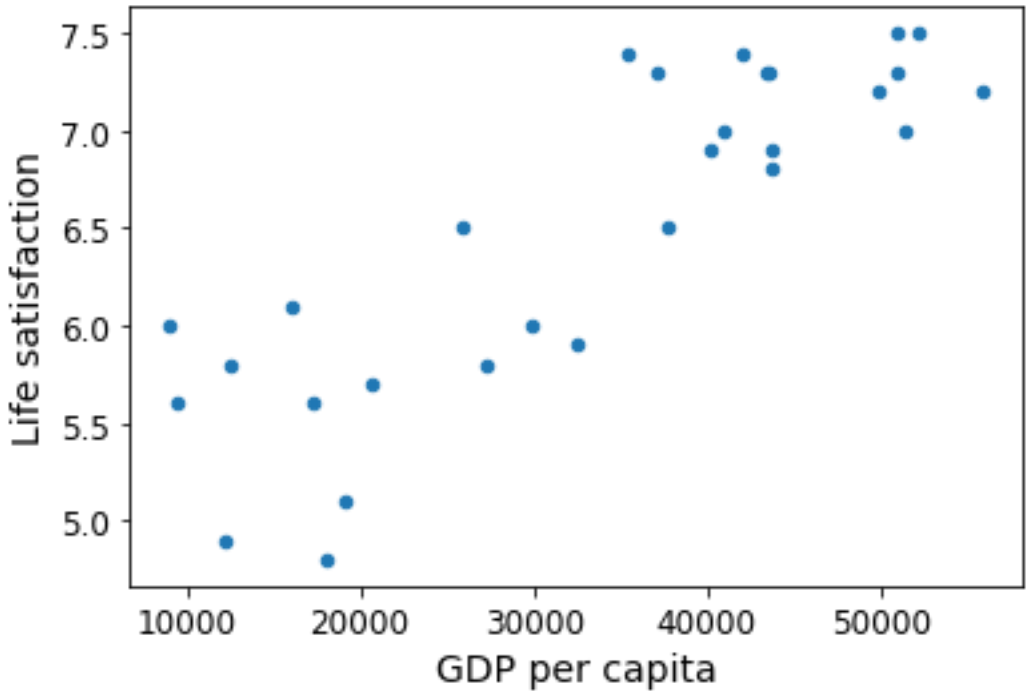
Readings: Technical Notes

- Distributions
- LOESS
- R^2 / ROC / AUC
- Gini coefficient
- Cross-validation (today) – method for determining hyperparameters, and how well your model can be generalized to new data
- Elastic Net and regularization (today): finding parsimonious models
  - Parsimonious: "frugal to the point of stinginess"
  - Related to feature engineering

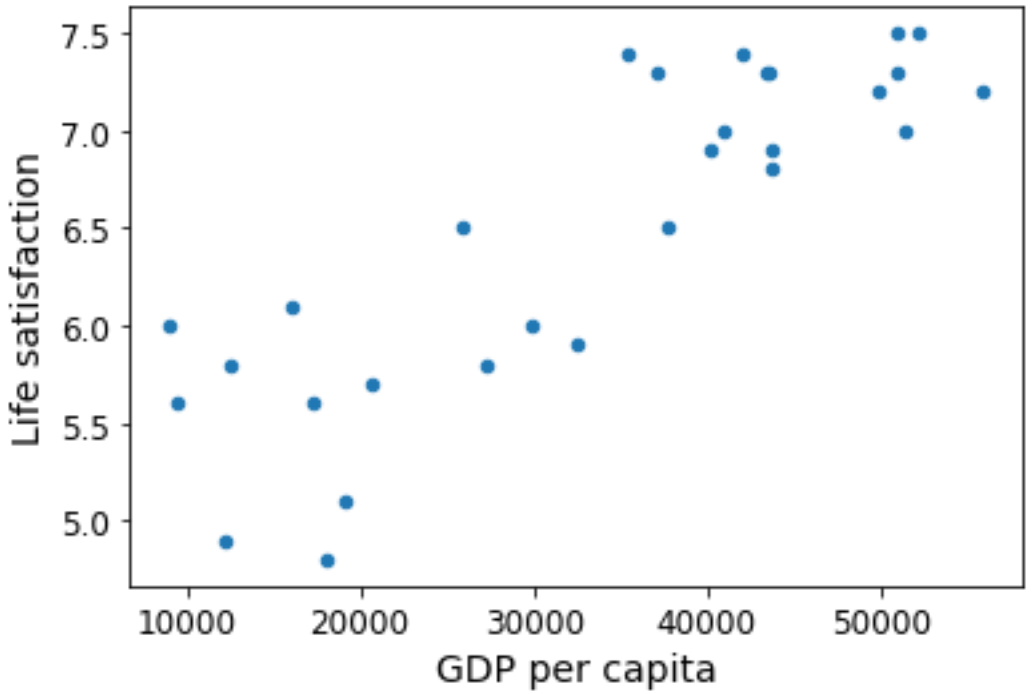
Other thoughts

- Independent vs dependent variables – how are relationships established?

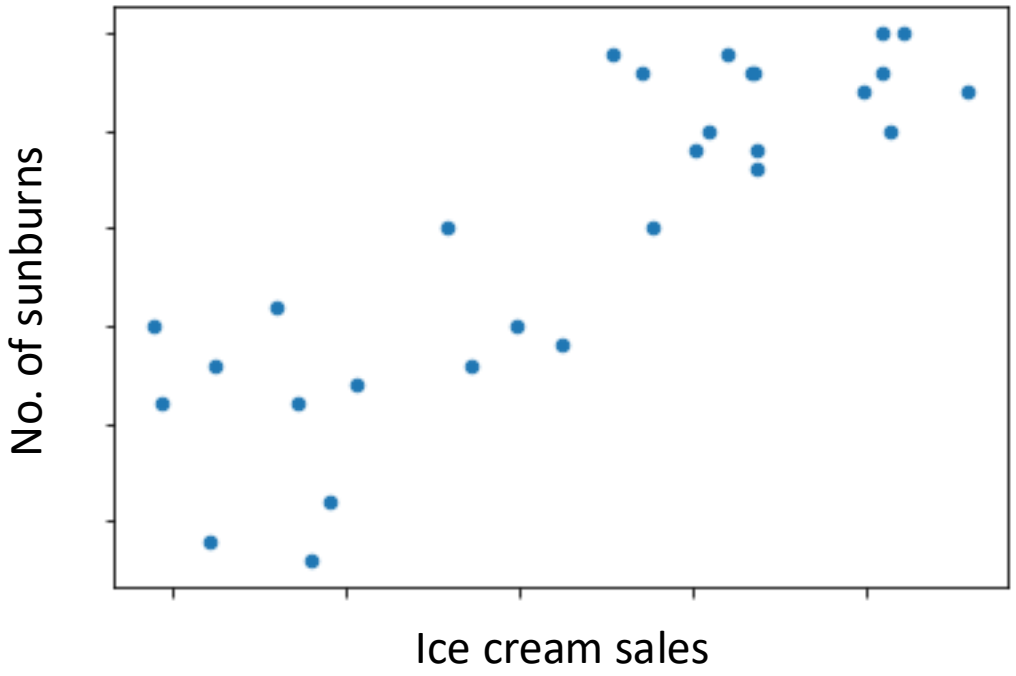
Week #1:

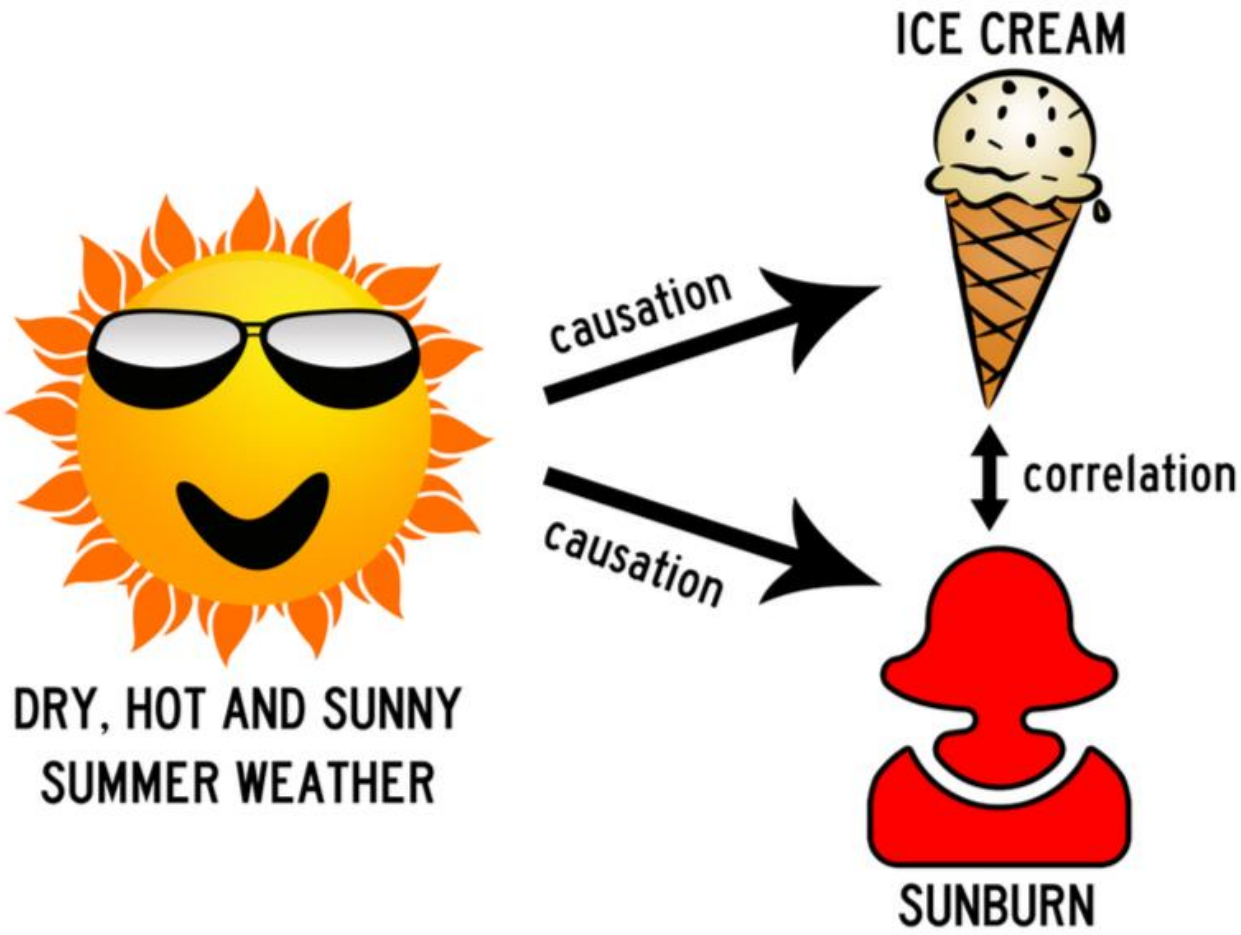

7.5
7.0
6.5
6.0
5.5
5.0
Life satisfaction
10000
20000
30000
40000
50000
GDP per capita

Week #1:



How about this?
Santa Monica Beach during
May 2022




ICE CREAM
causation
correlation
causation
DRY, HOT AND SUNNY
SUMMER WEATHER
SUNBURN

Other thoughts

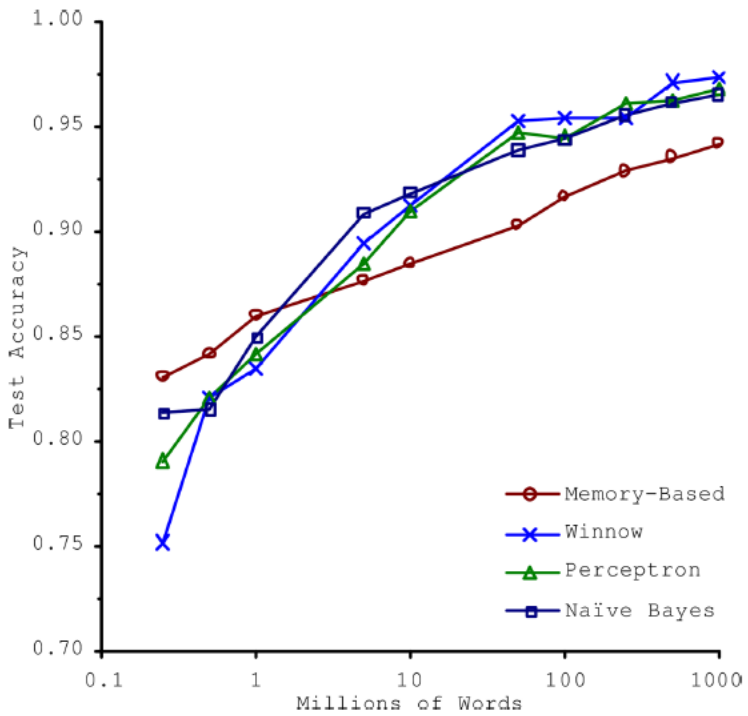
- Independent vs dependent variables – how are relationships established?
  - Causal Inference -> a different but related (and developing) field of inquiry
  - We can make any prediction we want with a model, but a model is just a model – machine learning is different than carrying out carefully controlled experiments

Notes

- Using people as subjects
  - Personally identifiable information, privacy, biases
- Sampling methods
  - iPhone users
- Measurement
  - Do phones actually completely capture activity levels?
- Target characteristic
  - How is wealth defined? Are there other ways to define it?

ML Challenges

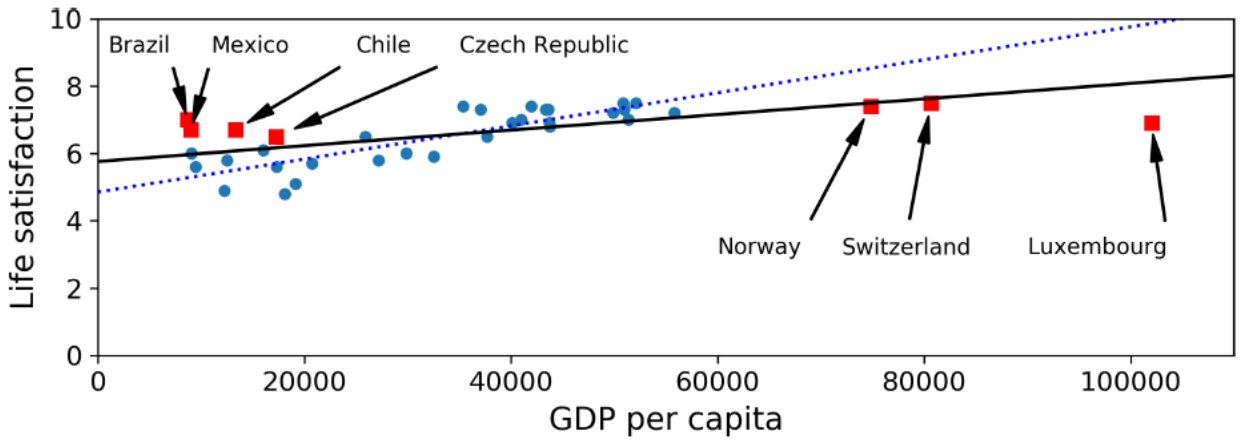
- Insufficient data



"Scaling to Very Very Large Corpora for Natural Language Disambiguation"
Michele Banko and Eric Brill, Microsoft Research, 2001
https://doi.org/10.3115/1073012.1073017

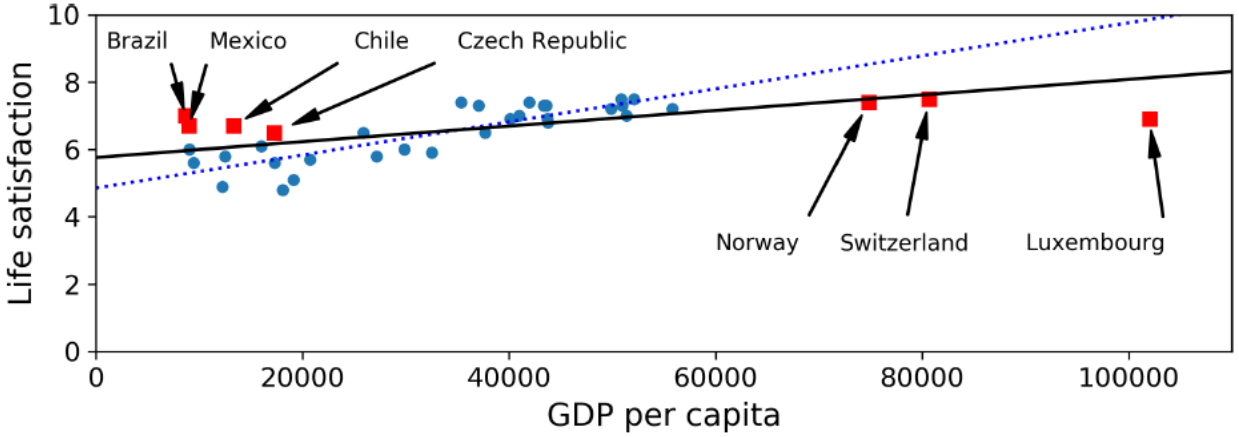
ML Challenges

- Insufficient data
- Nonrepresentative data



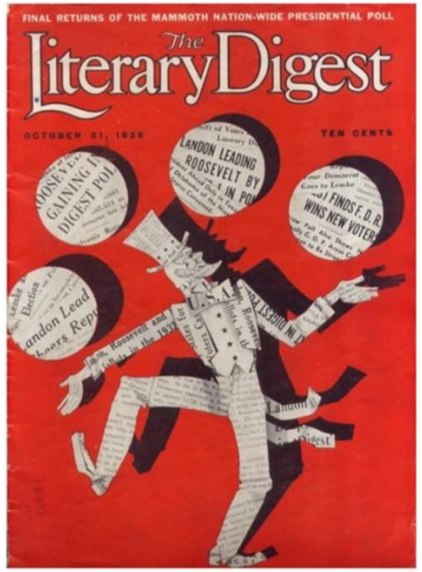
ML Challenges

- Insufficient data
- Nonrepresentative data
  - Data was too small -> sampling noise



ML Challenges

- Insufficient data
- Nonrepresentative data
  - Data was too small -> sampling noise
  - Data was sampled in a flawed way



25% of 2.4 million responses: Landon 57% vs Roosevelt 40%

Selection bias
-- Democratss less likely to have phones in 1936
-- Or drive a car
-- Or read literary digest

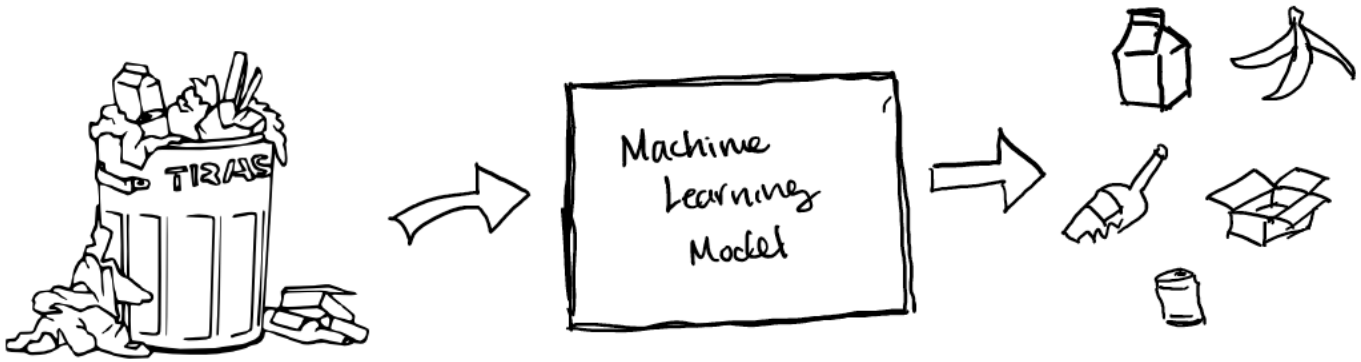
Non-response bias
-- Only 20% responded
-- Respondents favored Landon

ML Challenges

- Insufficient data
- Nonrepresentative data
- Poor-quality data
  - Errors, outliers, noise
  - Take personnel data: If 10% of sample doesn't have a degree recorded:
    - Ignore degree?
    - Fill in with most frequently occurring value? (say, bachelor's degree?)
    - Train one model with degree and another without it?

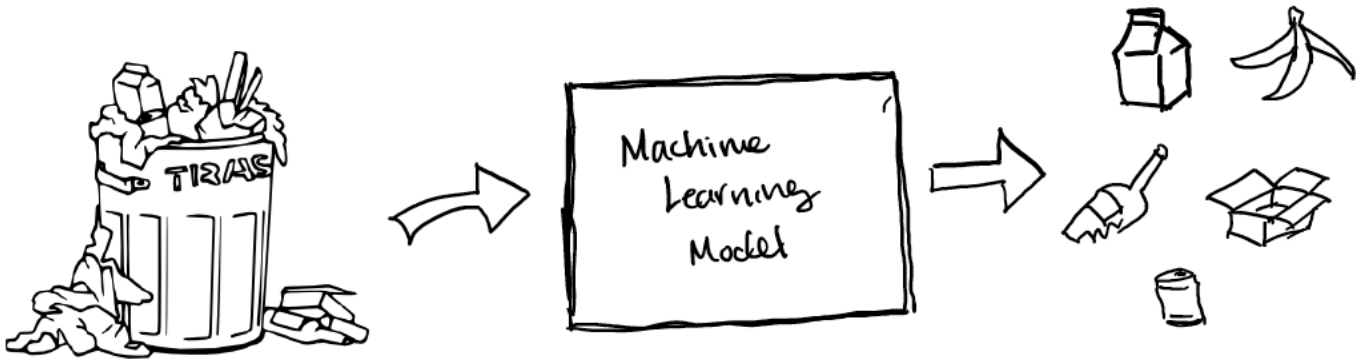
ML Challenges

- Insufficient data
- Nonrepresentative data
- Poor-quality data
- Irrelevant data



ML Challenges

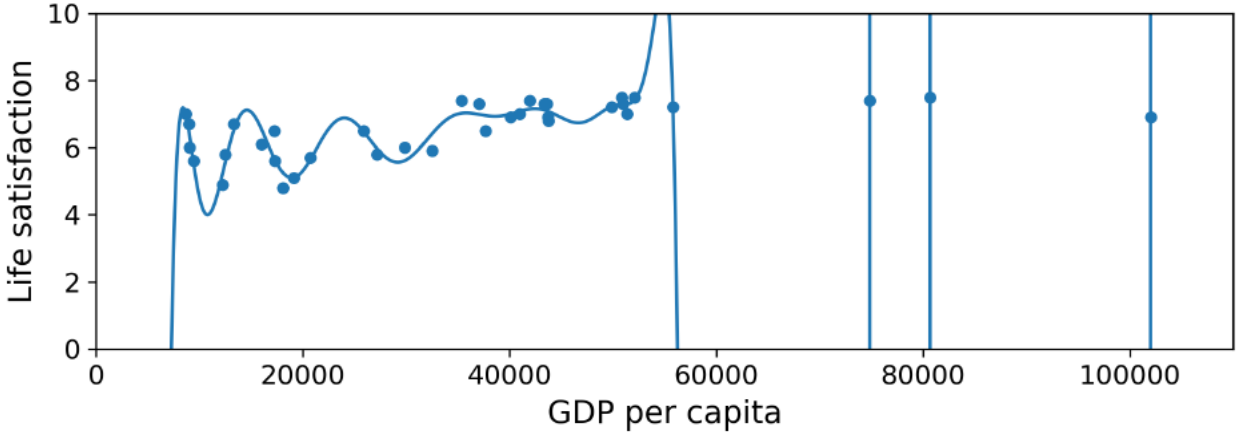
- Insufficient data
- Nonrepresentative data
- Poor-quality data
- Irrelevant data
  - Feature engineering
    - Select most useful features
    - Combine existing features
    - Get new features (data)



ML Challenges

- Insufficient data
- Nonrepresentative data
- Poor-quality data
- Irrelevant data
- Overfitting / underfitting

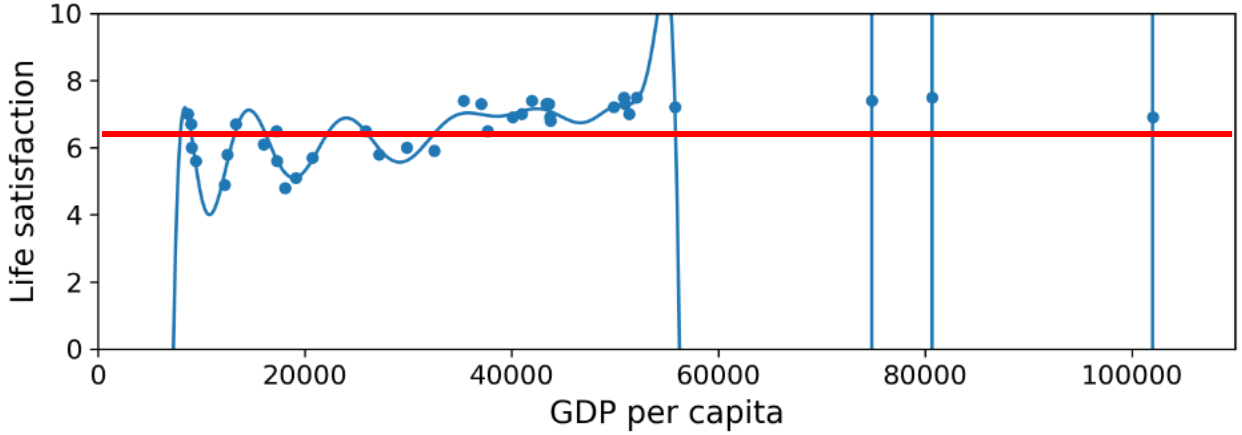
Overfitting performs very well on training data but doesn't generalize well



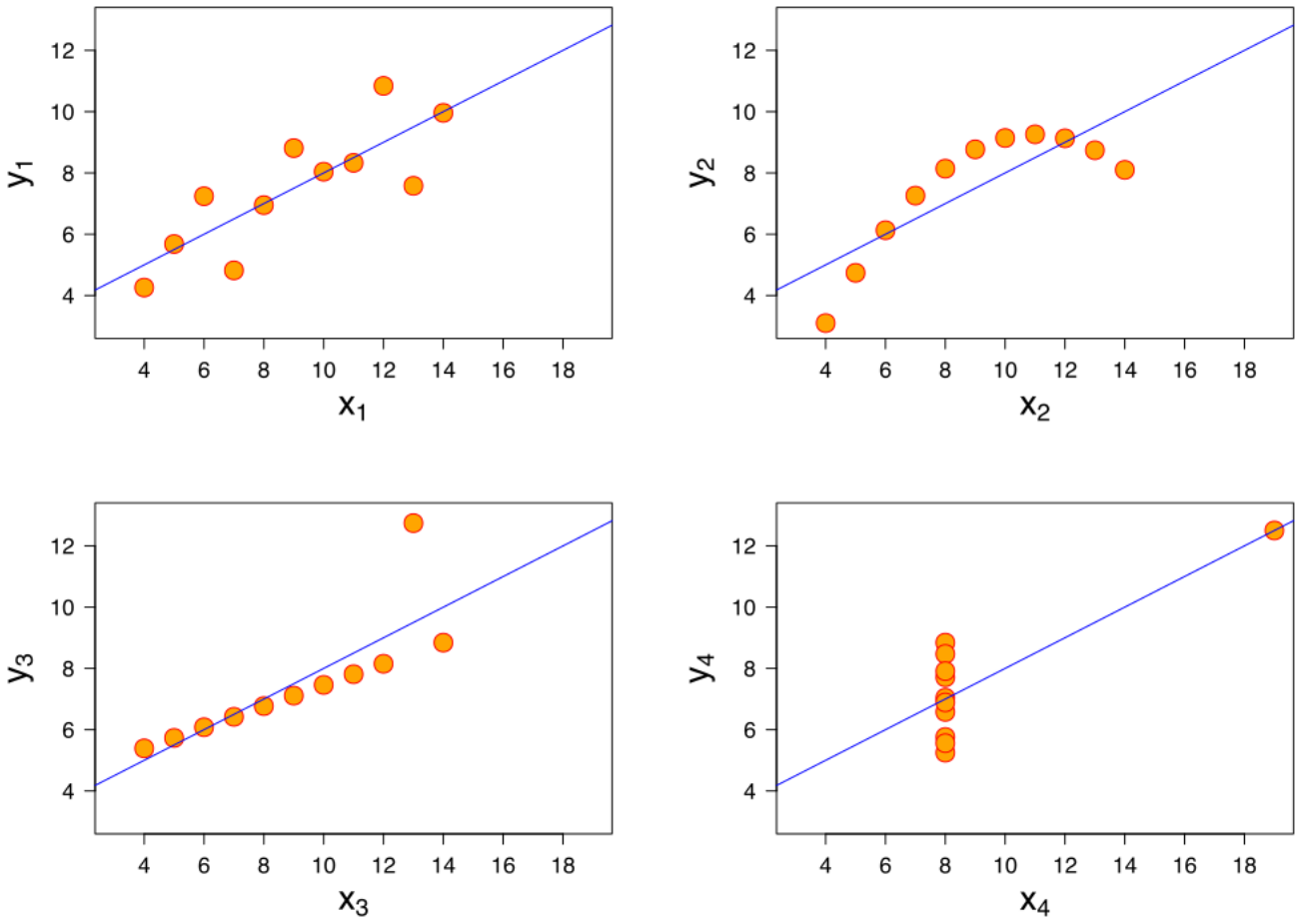
ML Challenges

- Insufficient data
- Nonrepresentative data
- Poor-quality data
- Irrelevant data
- Overfitting / underfitting

Underfitting occurs when the model is just too simple to capture the data's underlying structure

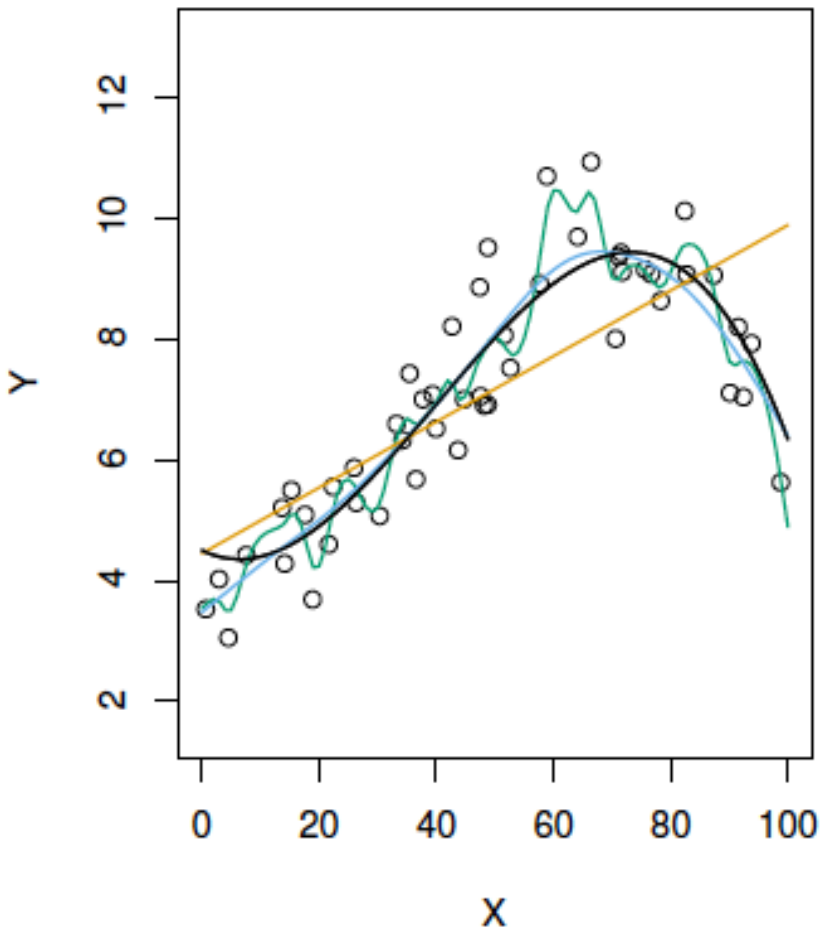


Blindly doing a linear fit



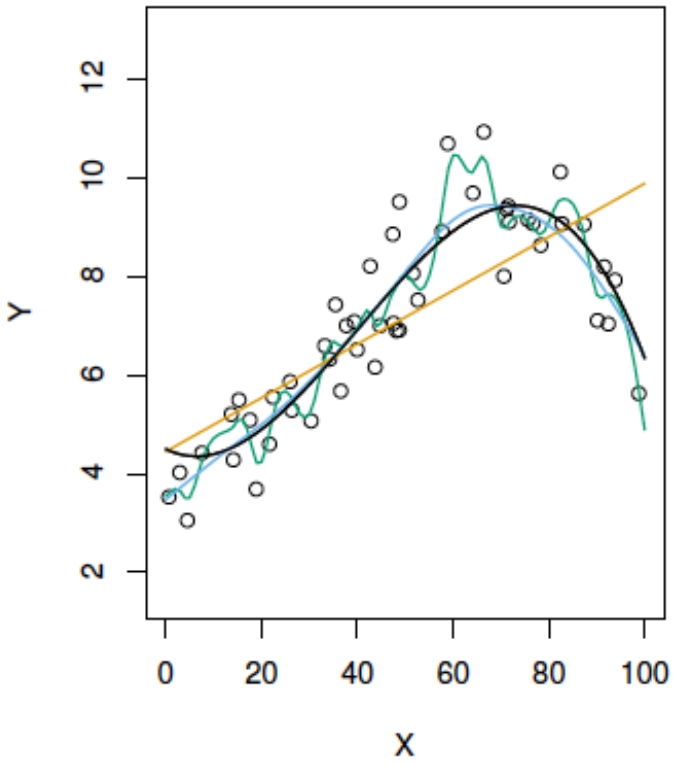
https://en.wikipedia.org/wiki/Anscombe%27s_quartet

ML Challenges: bias-variance trade-off



ESL

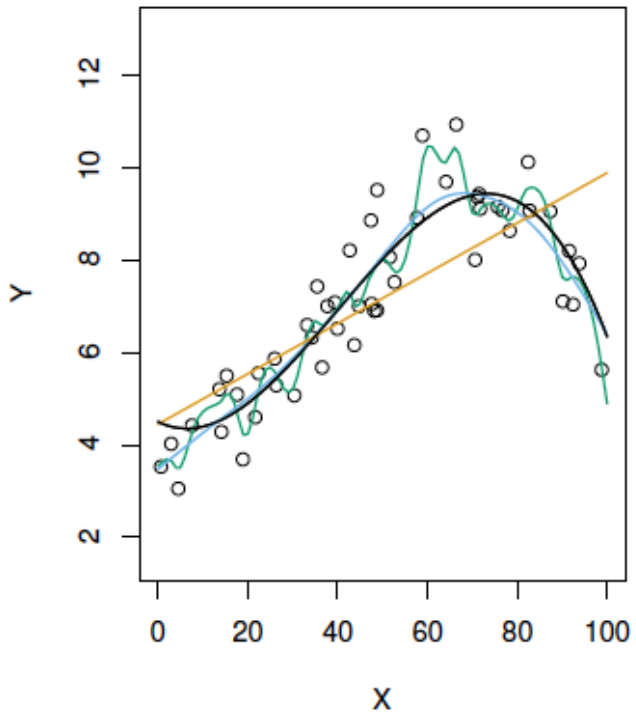
ML Challenges

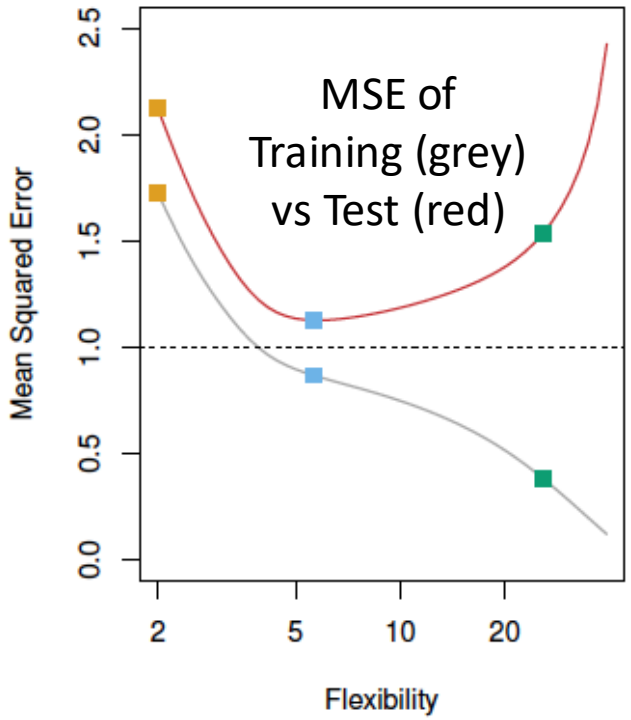


- Insufficient data
- Nonrepresentative data
- Poor-quality data
- Irrelevant data
- Overfitting / underfitting
  - Bias-variance trade-off $E\left(y_0 - \hat{f}(x_0)\right)^2 = \mathrm{Var}(\hat{f}(x_0)) + [\mathrm{Bias}(\hat{f}(x_0))]^2 + \mathrm{Var}(\epsilon)$
    - Bias: the error of approximating a complex real-life problem with a simple model
    - Variance: the amount our prediction would change if we estimated it with a different data set
      - If small changes to the data lead to big changes in predictions -> high variance

Testing your model, part I

- How can you tell whether you've underfit or overfit?
  - Train/test split
    - Example: 80/20 split
    - 80% of data to train your model
    - 20% of data to test how well your model generalizes to new data
    - Training error low but generalization error high -> overfitting





Code Example

Testing your model, part II

- How do we know what the best number of neighbors is for KNN?
  - This affects the learning algorithm, but it’s not a parameter that is optimized when running the algorithm

Testing your model, part II

- How do we know what the best number of neighbors is for KNN?
  - This affects the learning algorithm, but it's not a parameter that is optimized when running the algorithm
  - This is a *hyperparameter*

Testing your model, part II

- How do we know what the best number of neighbors is for KNN?
  - This affects the learning algorithm, but it's not a parameter that is optimized when running the algorithm
  - This is a *hyperparameter*
- Take a bunch of hyperparameter values, train your model for all of them, and compare how they perform on your test data!

Testing your model, part II

- How do we know what the best number of neighbors is for KNN?
  - This affects the learning algorithm, but it's not a parameter that is optimized when running the algorithm
  - This is a *hyperparameter*
- ~~Take a bunch of hyperparameter values, train your model for all of them, and compare how they perform on your test data!~~
- NO!
  - This might overfit too, this time by overfitting to your test data

Testing your model, part II

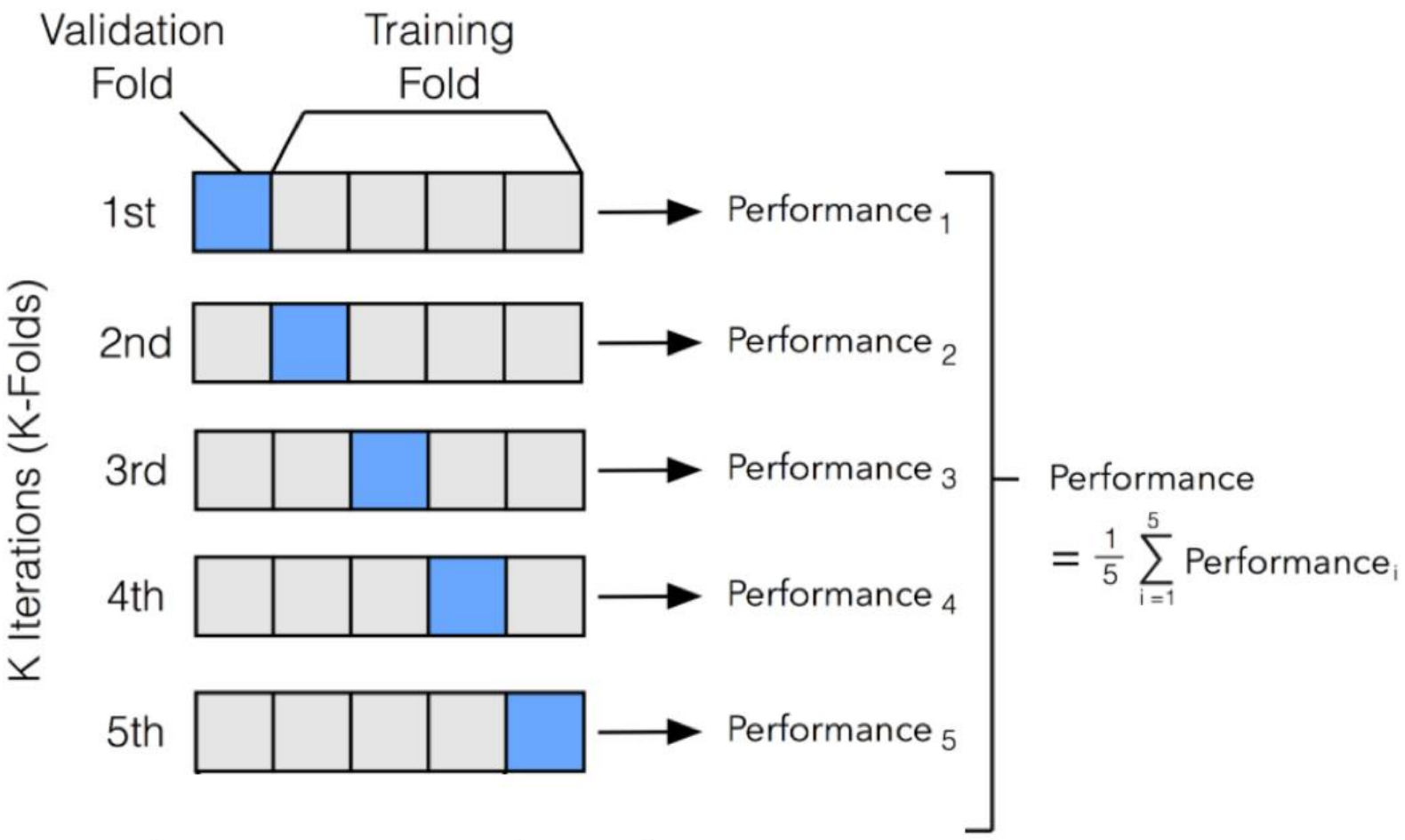
- Optimize hyperparameter values with a validation set
  - [train set + validation set] + test set
  - Once you've settled on their values, only then do you perform your final training, followed by testing on the final test set

Testing your model, part II

- Optimize hyperparameter values with a validation set
  - [train set + validation set] + test set
  - Once you've settled on their values, only then do you perform your final training, followed by testing on the final test set

- Cross-validation: use multiple samples of your training data to test and train your model over multiple iterations

K-folds Cross Validation (here with 5 folds)



https://ethen8181.github.io/machine-learning/model_selection/img/kfolds.png

Coding time

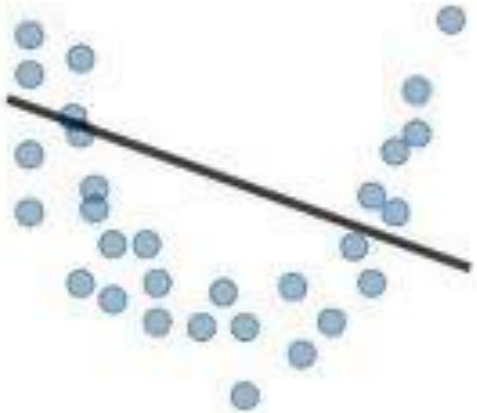
Testing your model, part II

- Optimize hyperparameter values with a validation set
  - [train set + validation set] + test set
  - Once you've settled on their values, only then do you perform your final training, followed by testing on the final test set

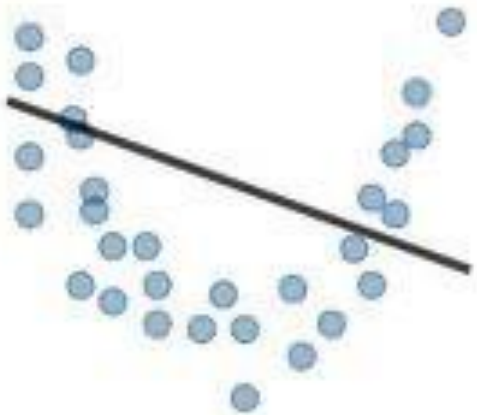
- Cross-validation: use multiple samples of your training data to test and train your model over multiple iterations

- Putting this in context
  - Cross-validation only yields meaningful results if the validation set and training set are drawn from the same population and only if biases are controlled.
  - Ex: stock prices over time, diagnosing disease risk across populations

ML Challenges: bias-variance trade-off

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Regression illustration			
Possible remedies	• Complexify model • Add more features • Train longer		• Perform regularization • Get more data

ML Challenges: bias-variance trade-off

	Underfitting	Just right	Overfitting
Symptoms	• High training error • Training error close to test error • High bias	• Training error slightly lower than test error	• Very low training error • Training error much lower than test error • High variance
Regression illustration			
Possible remedies	• Complexify model • Add more features • Train longer		• Perform regularization ???? • Get more data

ML Challenges

- Overfitting
  - Choose a simpler model
    - Fewer parameters in model (linear vs quadratic)
    - Fewer features
    - Constrain the model
  - Gather more data
  - Reduce data noise (errors, outliers, …)
- Underfitting
  - Use model with more parameters
  - Select better features
  - Reduce constraints on the model

Model-based Learning Example

- Overfitting
  - Choose a simpler model
    - Fewer parameters in model (linear vs quadratic)
    - Fewer features
    - Constrain the model
  - Gather more data
  - Reduce data noise (errors, outliers, …)
- Underfitting
  - Use model with more parameters
  - Select better features
  - Reduce constraints on the model

Feature Engineering

Model-based Learning Example

- Overfitting
  - Choose a simpler model
    - Fewer parameters in model (linear vs quadratic)
    - Fewer features
    - Constrain the model
  - Gather more data
  - Reduce data noise (errors, outliers, …)
- Underfitting
  - Use model with more parameters
  - Select better features
  - Reduce constraints on the model

Regularization

Slight Detour Back to Week #1:
Methods to Train a Linear Regression Model

Specific training example: Linear Model

- Hypothetical model:

$$y_{pred}(X) = \theta^T X$$

- How do we figure out what the best coefficients are?

Specific training example: Linear Model

- Hypothetical model:

$$y_{pred}(X) = \theta^T X$$

- How do we figure out what the best coefficients are?
  - Given our data, choose θ that gives the smallest possible mean squared error between the predictions and actual target values

Specific training example: Linear Model

- Hypothetical model:

$$y_{pred}(X) = \theta^T X$$

- How do we figure out what the best coefficients are?
  - Given our data, choose θ that gives the smallest possible mean squared error between the predictions and actual target values
- Mean squared error here is our cost function (loss function) J

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left(y_{pred}(X_i) - y_i\right)^2$$

Methods to get parameters:

1. Find the analytical solution

2. Iteratively minimize the cost function

3. Get someone else to do the work for us
   - Scikit-Learn

Method 1 to get parameters: Find analytical solution

- The case here actually has an exact analytical solution for min(J):

$$\frac{\partial J}{\partial \theta} = 0 \text{ when } \theta = (X^T X)^{-1} X^T y$$

- This will not happen very often, but definitely take advantage of it when it does

See for example https://eli.thegreenplace.net/2014/derivation-of-the-normal-equation-for-linear-regression/

Method 2 to get parameters: iteratively minimize the cost function

Specific training example: Linear Model

- Our cost function to minimize is:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left(y_{pred}(X_i) - y_i\right)^2 = \frac{1}{m}\sum_{i=1}^{m}((\theta_0 + \theta_1 X_i) - y_i)^2$$

- We have two coefficients, and we need to minimize J with respect to both:

$$\frac{\partial J}{\partial \theta_0} = \frac{2}{m}\sum_{i=0}^{m}\left(y_{pred}(X_i) - y_i\right)$$

$$\frac{\partial J}{\partial \theta_1} = \frac{2}{m}\sum_{i=0}^{m}\left(y_{pred}(X_i) - y_i\right) X_i$$

We can also get the minimum by iteratively minimizing the cost function

- Gradient descent algorithm:
  - Calculate the gradient: $\frac{\partial J}{\partial \theta_0} = \frac{2}{m}\sum_{i=0}^{m}\left(y_{pred}(X_i) - y_i\right)$; $\frac{\partial J}{\partial \theta_1} = \frac{2}{m}\sum_{i=0}^{m}\left(y_{pred}(X_i) - y_i\right) X_i$
  - Take a step in the opposite direction: $\theta_0 = \theta_0 - a\frac{\partial J}{\partial \theta_0}$; $\theta_1 = \theta_1 - a\frac{\partial J}{\partial \theta_1}$
  - Check the error and repeat as needed

We can also get the minimum by iteratively minimizing the cost function

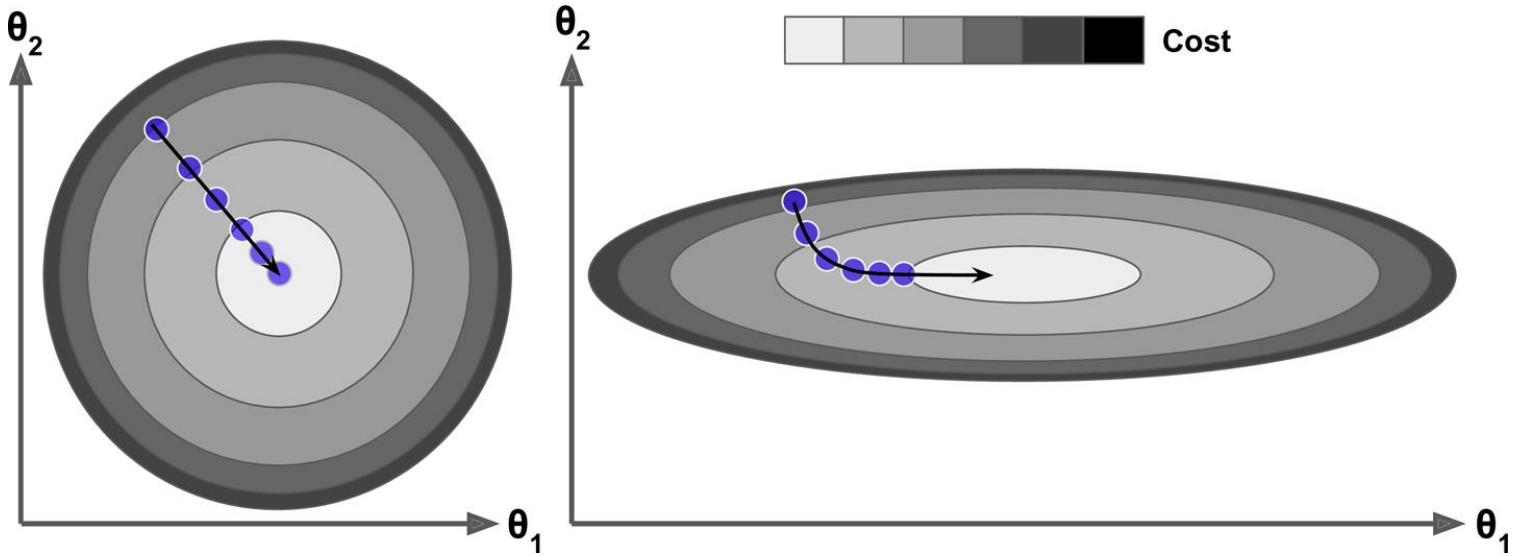
- Gradient descent algorithm:
  - Calculate the gradient: $\frac{\partial J}{\partial \theta_0} = \frac{2}{m}\sum_{i=0}^{m}\left(y_{pred}(X_i) - y_i\right)$; $\frac{\partial J}{\partial \theta_1} = \frac{2}{m}\sum_{i=0}^{m}\left(y_{pred}(X_i) - y_i\right) X_i$
  - Take a step in the opposite direction: $\theta_0 = \theta_0 - a\frac{\partial J}{\partial \theta_0}$; $\theta_1 = \theta_1 - a\frac{\partial J}{\partial \theta_1}$
  - Check the error and repeat as needed

There are three key ways to compute these and update the parameters:

- All data: Batch Gradient Descent
- One datum at a time: Stochastic Gradient Descent
- Intermediate batch sizes: Mini-Batch Gradient Descent

Batch Gradient Descent

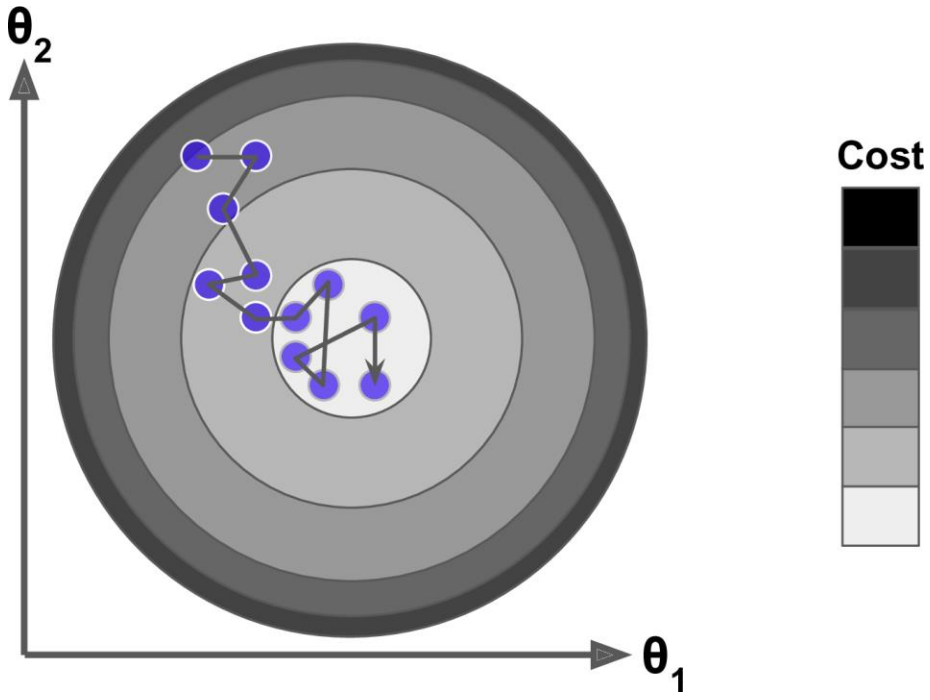
- Computationally expensive but descends right towards the minimum



- This also illustrates why it can be a good idea to scale the data to improve convergence time

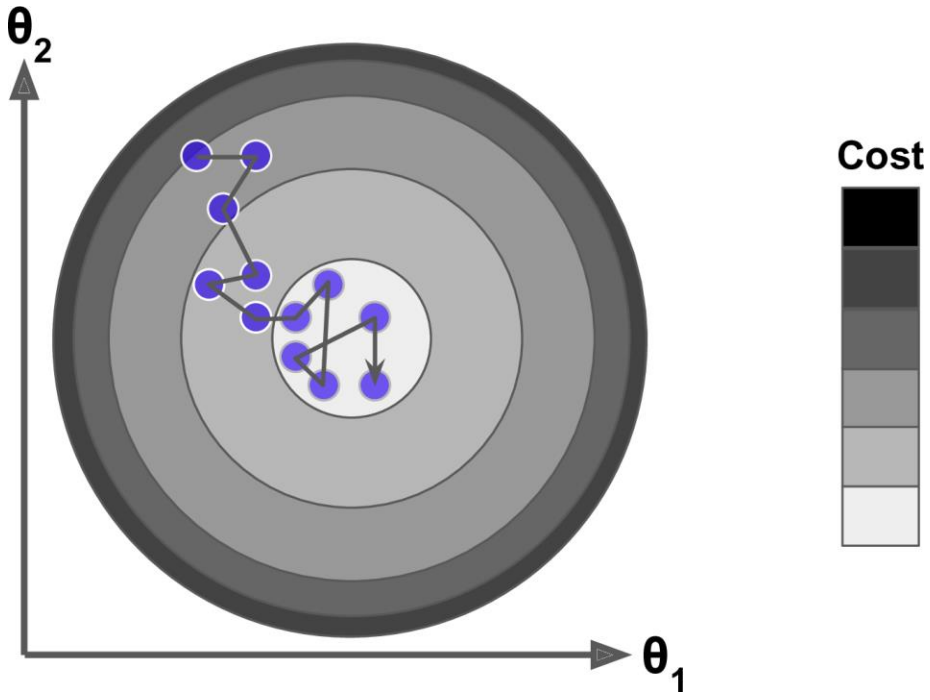
Stochastic Gradient Descent

- Much faster – works well for very large datasets
- But, more stochastic (random) evolution towards the minimum (which it never optimally reaches)
- But, it is less likely to get stuck in a local minimum
- Can use a “learning schedule” to reduce the learning rate over time
- A complete iteration over the size of the dataset is an *epoch*



Stochastic Gradient Descent

- Good idea to either pick data at random or shuffle after each epoch
- Since you never exactly get to the minimum, can set the iterations to end when you reach a certain tolerance.



Mini-Batch Gradient Descent

- Strikes a medium
  - Can get a performance boost
  - Progress is less stochastic than Stochastic Gradient Descent
  - May be harder to escape local minima

How does regularization fit into this?

1. Find the analytical solution

2. Iteratively minimize the cost function

3. Get someone else to do the work for us
   - Scikit-Learn

Addressing overfitting / underfitting

- Overfitting
  - Choose a simpler model
    - Fewer parameters in model (linear vs quadratic)
    - Fewer features
    - Constrain the model
  - Gather more data
  - Reduce data noise (errors, outliers, …)
- Underfitting
  - Use model with more parameters
  - Select better features
  - Reduce constraints on the model

Regularization

Regularization

- Constrain or simplify the model to make it more generalizable

Regularization

- Constrain or simplify the model to make it more generalizable
  - If you have a high-order polynomial, you can constrain it by simply reducing the order

Regularization

- Constrain or simplify the model to make it more generalizable
  - If you have a high-order polynomial, you can constrain it by simply reducing the order
- You can also modify the cost function
  - Add a parameter-dependent term to the cost function to constrain the parameter sizes

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(y_{pred}(X_i) - y_i)^2 + R(\theta)$$

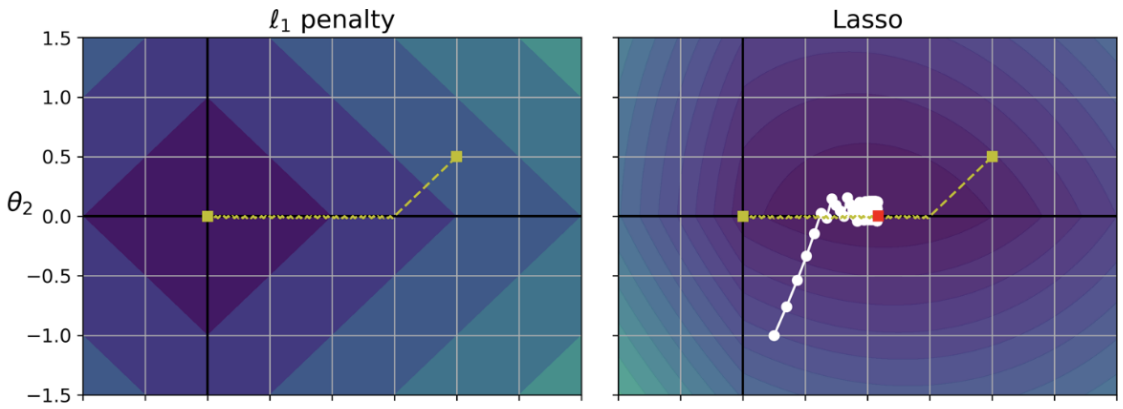
Lasso: $R(\theta) = \alpha \sum_{i=1}^{n} |\theta_i|$

Ridge: $R(\theta) = \frac{\alpha}{2} \sum_{i=1}^{n} \theta_i^2$

Elastic Net: $R(\theta) = r\alpha \sum_{i=1}^{n} |\theta_i| + \frac{(1-r)\alpha}{2} \sum_{i=1}^{n} \theta_i^2$

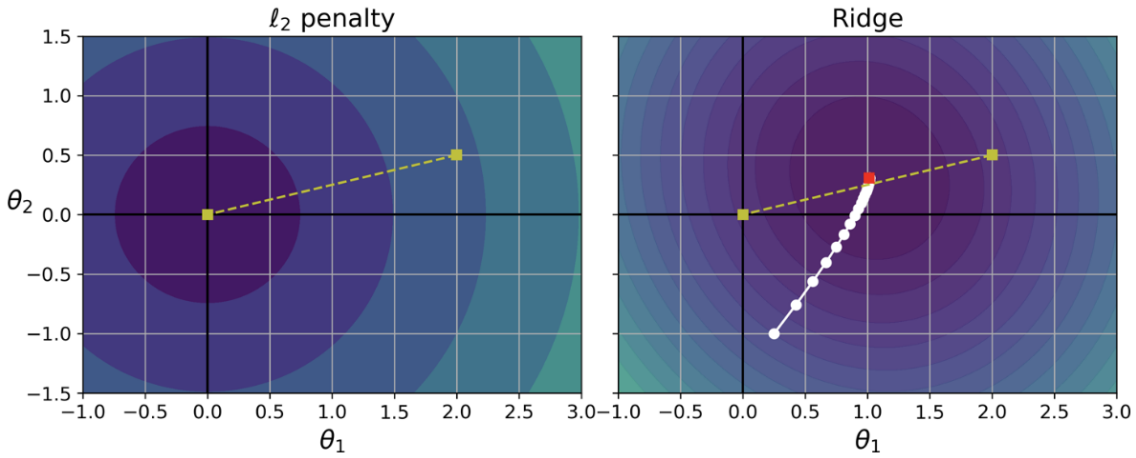
Lasso Regularization

- Lasso: Least Absolute Shrinkage and Selection Operator Regression
  - Uses the $\ell 1$ norm
    - $R(\theta) = \alpha \sum_{i=1}^{n} |\theta_i|$
    - $\partial R / \partial \theta = \alpha \sum_{i=1}^{n} (\pm 1)$
    - sklearn: “SGDRegressor(penalty="l1")”
  - Tends to zero out the least important features
  - Also tends to bounce around the global optimum
    - gradient is never zero, only -1 or +1
    - may behave erratically when the number of features is larger than the number of training instances, or when features are strongly correlated



Ridge Regularization

- Ridge
  - Uses the $\ell 2$ norm
    - $R(\theta) = \frac{\alpha}{2}\sum_{i=1}^{n}\theta_i^2$
    - $\partial R/\partial\theta = \alpha\sum_{i=1}^{n}(\theta_i)$
    - sklearn: “SGDRegressor(penalty="l2")”)
  - Convergence is aided by the gradient getting smaller as parameters approach the optimum



Ridge vs Lasso vs Elastic Net Regularization

- Advice on what to use?
  - Ridge is a good default that handles multicollinearity
  - Lasso or Elastic Net are useful to dismiss some of the least important features
  - Elastic Net is usually better, as it grabs benefits of both regularization methods
    - And doesn't suffer from as much erratic behavior near the optimum
    - May involve experimenting with hyperparameters

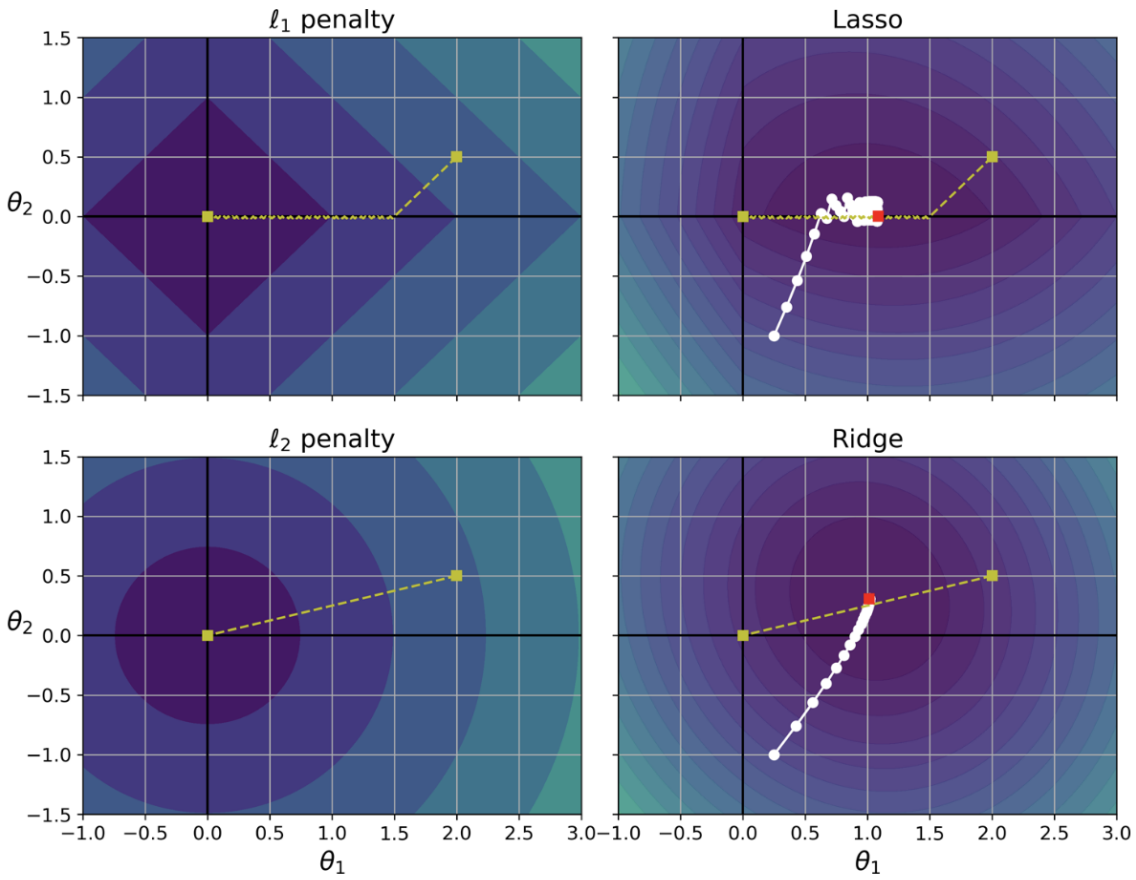


Figure 4-19. Lasso versus Ridge regularization

Another (non-model) method for regularization

- Stop the iterative learning algorithm when the validation error is a minimum

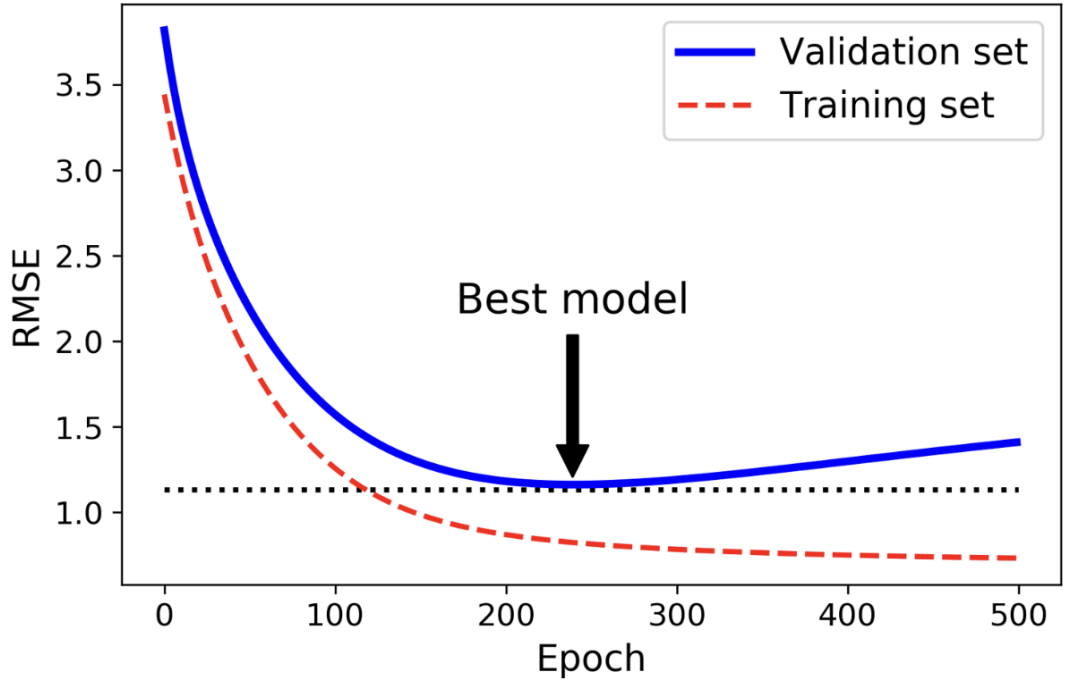
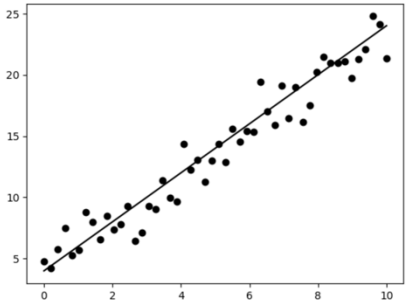


Figure 4-20. Early stopping regularization

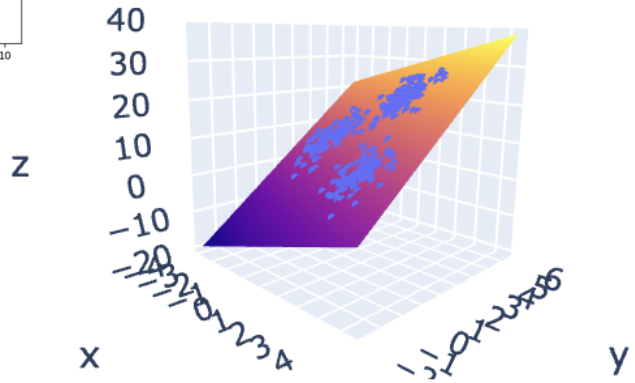
Code Time
Brief Example of Regularization
Housing Exercise

Linear vs Multi-linear vs Polynomial Regression

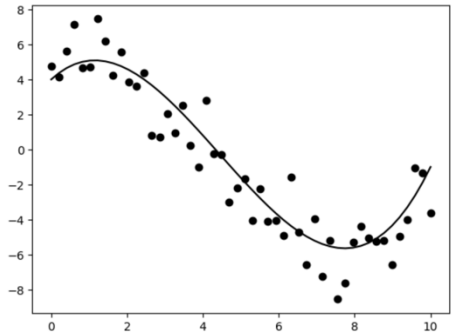
- Simple linear regression
  - $y = a_0 + a_1 x$



- Multi-linear regression
  - $y = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_3 + \cdots$



- Polynomial regression
  - $y = a_0 + a_1 x^1 + a_2 x^2 + a_3 x^3 + \cdots$
  - Can be treated like multi-linear regression!
  - $y = a_0 + a_1 x_1 + a_2 x_2 + a_3 x_3 + \cdots$



Code Time
Multilinear and Polynomial Regression
With Regularization